МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ

ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных Технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 Программная Инженерия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

«Разработка компилятора PVR-2025»

Выполнил студент Потребко Вадим Русланович
(Ф.И.О.)

Руководитель проекта преп.-стажер Лыкова Юлия Дмитриевна
(учен. степень, звание, должность, подпись, Ф.И.О.)

Заведующий кафедрой к.т.н., доц. Смелов В.В.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Консультанты преп.-стажер Лыкова Юлия Дмитриевна
(учен. степень, звание, должность, подпись, Ф.И.О.)

(учен. степень, звание, должность, подпись, Ф.И.О.)

Нормоконтролер ст.преп. Наркевич Аделина Сергеевна
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой

Минск, 2025

Оглавление

Введение 5
1 Спецификация языка программирования 6
1.1 Характеристика языка программирования 6
1.2 Определение алфавита языка программирования 6
1.3 Применяемые сепараторы 6
1.4 Применяемые кодировки 7
1.5 Типы данных 7
1.6 Преобразование типов данных 8
1.7 Идентификаторы 8
1.8 Литералы 9
1.9 Инициализация данных 10
1.10 Инициализация данных 11
1.11 Инструкции языка 11
1.12 Операции языка 12
1.13 Выражения и их вычисления 13
1.14 Конструкции языка 13
1.15 Область видимости идентификаторов 15
1.16 Семантические проверки 15
1.17 Распределение оперативной памяти на этапе выполнения 15
1.18 Стандартная библиотека и её состав 15
1.19 Ввод и вывод данных 16
1.20 Точка входа 16
1.21 Препроцессор 17
1.22 Соглашения о вызовах 17
1.23 Объектный код 17
1.24 Классификация сообщений транслятора 17
1.25 Контрольный пример 17
2. Структура транслятора 18
2.1 Компоненты транслятора, их назначение и принципы взаимодействия 18
2.2 Перечень входных параметров 18

2.3 Протоколы, формируемые транслятором 19
3. Разработка лексического анализатора 20
3.1 Структура лексического анализатора 20
3.2 Входные и выходные данные лексического анализатора 20
3.3 Параметры лексического анализатора 20
3.4 Алгоритм лексического анализатора 20
3.5 Контроль входных символов 21
3.6 Принцип обработки ошибок 21
3.7 Удаление избыточных символов 22
3.8 Перечень ключевых слов 23
3.9 Структура и перечень сообщений лексического анализатора 24
3.10 Принцип обработки ошибок 24
3.11 Контрольный пример 24
4. Разработка синтаксического анализатора 25
4.1 Структура синтаксического анализатора 25
4.2 Контекстно свободная грамматика, описывающая синтаксис языка 25
4.3 Построение конечного магазинного автомата 26
4.4 Основные структуры данных 27
4.5 Описание алгоритма синтаксического разбора 27
4.6 Параметры синтаксического разбора 27
4.7 Структура и перечень сообщений синтаксического анализатора 27
4.8 Принцип обработки ошибок 28
4.9 Контрольный пример 28
5. Разработка семантического анализатора 29
5.1 Структура семантического анализатора 29
5.2 Функции семантического анализатора 29
5.3 Структура и перечень сообщений семантического анализатора 29
5.4 Принцип обработки ошибок 30
5.5 Контрольный пример 30
6. Преобразование выражений 31
6.1 Выражения, допускаемые языком 31
6.2 Польская запись и принцип ее построения 31

6.3 Программная реализация обработки выражений 32
6.4 Контрольный пример 32
7. Генерация кода 34
7.1 Структура генератора кода 34
7.2 Представление типов данных в оперативной памяти 34
7.3 Статическая библиотека 35
7.4 Особенности алгоритма генерации кода 35
7.5 Параметры управляющие генерацией кода 36
7.6 Контрольный пример 36
8. Тестирование транслятора 37
8.1 Общие положения 37
8.2 Результаты тестирования 37
Заключение 40
Список использованной литературы 41
Приложение А 42
Приложение Б 44
Приложение В 45
Приложение Г 52
Приложение Д 53
Приложение Е 67
Приложение Ж 70

Введение

Целью курсовой работы является разработка языка программирования PVR-2025 и транслятора для него. Язык программирования PVR-2025 является языком общего назначения и позволяет использовать базовые математические операции, вывод данных и с их помощью реализовывать программы. Основная задача транслятора создание понятной компьютеру программу на языке PVR-2025. Трансляция языка PVR-2025 происходит в код на низкоуровневом языке JavaScript. Для курсовой работы были определены следующие задачи:

- разработка спецификации языка программирования PVR-2025;
- разработка и реализация структуры транслятора;
- разработка и реализация структуры лексического анализатора;
- разработка и реализация структуры синтаксического анализатора;
- разработка и реализация структуры семантического анализатора;
- генерация исходного кода в код на языке JavaScript;

Решения выше поставленных задач представлены в соответствующих главах курсовой работы, а именно:

В главе 1 описывается спецификация языка программирования.

В главе 2 представлена структура транслятора, перечень и содержимое обрабатываемых транслятором протоколов.

В главе 3 представлена структура лексического анализатора, представлены описания таблицы лексем и таблицы идентификаторов, а также перечень и содержимое ошибок, обрабатываемых лексическим анализатором.

В главе 4 представлена структура синтаксического анализатора, описывается осуществление синтаксического разбора текста, создание дерева разбора на основе таблицы лексем, а также перечень и содержимое ошибок, обрабатываемых синтаксическим анализатором.

В главе 5 представлено описание семантического анализатора, его работы, а также перечень и содержимое ошибок, обрабатываемых семантическим анализатором.

В главе 6 представлено описание преобразования выражений, допускаемых языком, а также результаты преобразования выражений в обратную польскую запись.

В главе 7 представлена генерация кода в язык JavaScript.

В главе 8 описан процесс тестирования транслятора.

1 Спецификация языка программирования

1.1 Характеристика языка программирования

Язык программирования PVR -2025 является высокоуровневым, строго типизированным языком, который транслируется в язык JavaScript. Язык дает возможность пользователю определять собственные процедуры и функции для их дальнейшего использования в программном коде.

1.2 Определение алфавита языка программирования

Для записи исходного текста на языке программирования PVR-2025 используются символы таблицы A. Windows-1251 — латинские буквы {A, B, C, …, Z, a, b, c, …, z} в верхнем и нижнем регистрах, цифры {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, и специальные символы: {+, -, _, %, #, (,), :, ?, !, /, *, &, =, ;, ‘ , ’ }. Использование других символов в программном коде недопустимо.

1.3 Применяемые сепараторы

Сепараторы являются лексемами, назначение которых разделение лексем в коде. В языке PVR-2025 применяются следующие символы сепараторы, представленные ниже в таблице 1.1.

Таблица 1.1 - Применяемые сепараторы

Сепаратор	Назначение сепаратора
;	Разделитель инструкций
{ }	Программный блок
( )	Параметры функций, приоритетность операций в выражениях
Пробел	Служит для разделения программных конструкций. Допускается везде, кроме идентификаторов и ключевых слов
,	Разделитель параметров в функции
+, -, *, /, %	Арифметические операторы
>, <, ~, !	Операторы сравнения

Применение неуказанных в таблице сепараторов не допускается.

1.4 Применяемые кодировки

В языке программирования PVR-2025 используются символы кодировки Windows-1251 для написания исходного текста. Ниже представлена таблица кодов Windows-1251

Таблица кодов Windows-1251:

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	NUL 0000	STX 0001	SOT 0002	ETX 0003	EOT 0004	ENQ 0005	ACK 0006	BEL 0007	BS 0008	HT 0009	LF 000A	VT 000B	FF 000C	CR 000D	SO 000E	SI 000F
10	DLE 0010	DC1 0011	DC2 0012	DC3 0013	DC4 0014	NAK 0015	SYN 0016	ETB 0017	CAN 0018	EM 0019	SUB 001A	ESC 001B	FS 001C	GS 001D	RS 001E	US 001F
20	SP 0020	! 0021	" 0022	# 0023	$ 0024	% 0025	& 0026	' 0027	(0028	) 0029	* 002A	+ 002B	, 002C	- 002D	. 002E	/ 002F
30	0 0030	1 0031	2 0032	3 0033	4 0034	5 0035	6 0036	7 0037	8 0038	9 0039	: 003A	; 003B	< 003C	= 003D	> 003E	? 003F
40	@ 0040	A 0041	B 0042	C 0043	D 0044	E 0045	F 0046	G 0047	H 0048	I 0049	J 004A	K 004B	L 004C	M 004D	N 004E	O 004F
50	P 0050	Q 0051	R 0052	S 0053	T 0054	U 0055	V 0056	W 0057	X 0058	Y 0059	Z 005A	[005B	\ 005C	] 005D	^ 005E	_ 005F
60	` 0060	a 0061	b 0062	c 0063	d 0064	e 0065	f 0066	g 0067	h 0068	i 0069	j 006A	k 006B	l 006C	m 006D	n 006E	o 006F
70	p 0070	q 0071	r 0072	s 0073	t 0074	u 0075	v 0076	w 0077	x 0078	y 0079	z 007A	{ 007B	| 007C	} 007D	~ 007E	DEL 007F
80	Ђ 0402	Ѓ 0403	‚ 201A	ѓ 0453	„ 201E	… 2026	† 2020	‡ 2021	€ 20AC	‰ 2030	Љ 0409	‹ 2039	Њ 040A	Ќ 040C	Ћ 040B	Џ 040F
90	ђ 0452	‘ 2018	’ 2019	“ 201C	” 201D	• 2022	– 2013	— 2014		™ 2122	љ 0459	› 203A	њ 045A	ќ 045C	ћ 045B	џ 045F
A0	NBSP 00A0	Ў 040E	ў 045E	Ј 0408	¤ 00A4	Ґ 0490	¦ 00A6	§ 00A7	Ё 0401	© 00A9	Є 0404	« 00AB	¬ 00AC	- 00AD	® 00AE	Ї 0407
B0	° 00B0	± 00B1	І 0406	і 0456	ґ 0491	µ 00B5	¶ 00B6	· 00B7	ё 0451	№ 2116	є 0454	» 00BB	ј 0458	Ѕ 0405	ѕ 0455	ї 0457
C0	А 0410	Б 0411	В 0412	Г 0413	Д 0414	Е 0415	Ж 0416	З 0417	И 0418	Й 0419	К 041A	Л 041B	М 041C	Н 041D	О 041E	П 041F
D0	Р 0420	С 0421	Т 0422	У 0423	Ф 0424	Х 0425	Ц 0426	Ч 0427	Ш 0428	Щ 0429	Ъ 042A	Ы 042B	Ь 042C	Э 042D	Ю 042E	Я 042F
E0	а 0430	б 0431	в 0432	г 0433	д 0434	е 0435	ж 0436	з 0437	и 0438	й 0439	к 043A	л 043B	м 043C	н 043D	о 043E	п 043F
F0	р 0440	с 0441	т 0442	у 0443	ф 0444	х 0445	ц 0446	ч 0447	ш 0448	щ 0449	ъ 044A	ы 044B	ь 044C	э 044D	ю 044E	я 044F

Рисунок 1.1 — Таблица кодов Windows-1251

Использование символов, не принадлежащих таблице кодов Windows-1251 в коде недопустимо.

1.5 Типы данных

В языке программирования PVR -2025 поддерживается целочисленный и строковый типы данных. Они являются фундаментальными типами данных в языке. Ниже представлены типы данных языка программирования PVR-2025 и их описание.

Таблица 1.2 — поддерживаемые языком программирования PVR-2025 типы данных и их описание

Тип данных	Описание
Целочисленный (short)	Тип short — целочисленный базовый тип данных. По умолчанию значение инициализируется нулём; размер составляет 1 байт, а допустимый диапазон — от −128 до 127. Для записи значений допускаются десятичные цифры и знак минуса; дополнительно можно использовать обозначения h (восьмеричная запись) и b (двоичная запись). Над значениями этого типа поддерживаются арифметические операции (сложение, вычитание, умножение, целочисленное деление и взятие остатка), а также операции сравнения: больше/меньше, больше/меньше или равно, равно и не равно.
Строковый (string)	Строковый тип. Инициализация по умолчанию: строка нулевой длины; размер в байтах: до 16 байтов, каждый символ занимает по одному байту; допускаемые символы: все символы языка, заключенные в одинарные кавычки (’ ’);

В языке PVR-2025 применяются только определенные языком типы данных. Создание пользовательских типов на основе фундаментальных не предусматривается.

1.6 Преобразование типов данных

В языке программирования PVR -2025 не допускается преобразование типов данных.

1.7 Идентификаторы

Идентификаторы являются уникальными именами для переменных, функций и параметров функций. Зарезервированные идентификаторы не предусмотрены. Имя идентификаторов в языке PVR-2025 не должно совпадать с ключевыми словами языка и с именами функций стандартной библиотеки, также идентификаторы не должны дублироваться.

Имя идентификатора составляется по следующим образом:

- состоит из символов латинского алфавита {a, b, c, …, z} d нижнем регистре;

- идентификатор не может содержать цифры, буквы в верхнем регистре, символ подчеркивания и другие символы;

- идентификатор может содержать от 1 до 10 символов.

Форма Бэкуса-Наура для записи идентификаторов:

<буква> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z

<идентификатор> ::= {/ <буква> /}

Примеры правильных идентификаторов:
- a;
- ab;

Примеры неправильных идентификаторов:
- 1ab;
- abcdefgi123.

1.8 Литералы

В языке существует 2 типа литералов: целого и строкового типов. Краткое описание литералов представлено в таблице 1.5.

Таблица 1.5 - Описание литералов

Тип литерала	Описание литерала
Целые (десятичные)	Десятичный целый литерал представляет собой последовательность цифр {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} без встроенных пробелов и разделителей. Допускается знак минус - перед числом для задания отрицательных значений. Значения целого типа по умолчанию инициализируются нулём. Диапазон допустимых значений для целых литералов в языке: от -128 до 127. Примеры правильных десятичных литералов: 0, 5, -12, 127. Примеры неправильных литералов: 1 0, 12-3, +5.
Целые (восьмеричные)	Восьмеричный целый литерал используется для представления чисел в системе счисления с основанием 8 и состоит из ведущего символа 0, за которым следуют цифры диапазона {0, 1, 2, 3, 4, 5, 6, 7}. Для записи отрицательных восьмеричных значений перед первой цифрой помещается символ N, обозначающий знак минус в принятой нотации. Внутри восьмеричного литерала не допускаются цифры 8 и 9, а также любые дополнительные символы. После преобразования восьмеричный литерал также должен попадать в диапазон от -128 до 127. Примеры правильных восьмеричных литералов: 010, 07, N077. Примеры неправильных литералов: 08, 0A7, N08, 0109.

Окончание таблицы 1.5

Строковые	Строковый литерал описывает последовательность символов алфавита языка, заключённую в одинарные кавычки ''. По умолчанию переменные строкового типа инициализируются пустой строкой, максимальная длина строкового литерала ограничена 255 символами. Внутри литерала допускаются буквы, цифры и знаки препинания, за исключением одинарной кавычки, которая используется как ограничитель. Примеры правильных строковых литералов: 'hello', '12q12'. Примеры неправильных литералов: hello, '12q12”, строка с незакрытой кавычкой.

Правила описание литералов языка PVR-2025 можно задать в форме Бэкуса-Наура следующим образом:

```
<литерал>::=<целое_число>|<строка>

<целое_число>::=<десятичное_число>|<восьмеричное_число>

<десятичное_число>::=['']<десятичная_цифра><обозначение_десятичного>

<обозначение_десятичного >::=|<десятичная_цифра><обозначение_дес.>

<восьмеричное_число>::=['N']'0'<восьмеричная_цифра><обозначение_восьм.>

<обозначение_восьмеричного_числа>::=|<восьмеричная_цифра><обозначение_во
сьм.>

<десятичная_цифра>::=0|1|2|3|4|5|6|7|8|9

<восьмеричная_цифра>::=0|1|2|3|4|5|6|7

<обозначение_строки> ::= { <символ_строки> }

<строка>::=<обозначение_строки>{<символ>}<обозначение_строки>

<символ>::="                                "|!|"|#|$|%|&|'|"("|")"|*|+|,|-
|.|/|0|1|2|3|4|5|6|7|8|9|:|;|<|=|>|?|@|A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S
|T|U|V|W|X|Y|Z|"["|"]"|^|_|`|a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x
|y|z|"{"|"|"|"}"|~
```

1.9 Объявления данных

Язык программирования PVR-2025 предполагает объявление переменной и ее типа до использования этой переменной. Объявление переменной допустимо только внутри функции. Областью видимости переменной является блок функции,

в которой она определена. Допускается объявление переменной без инициализации. Конструкция для объявления переменных имеет следующий вид:

```
Let <тип данных> <идентификатор>;
<идентификатор>=<литерал>|<идентификатор>;
```

Где let является ключевым словом для объявления переменных. Для инициализации используется оператор присваивания "=". Не допускается инициализация значением типа данных отличного от типа данных объявленного для переменной, это вызовет ошибку.

1.10 Инициализация данных

Инициализация данных при объявлении переменной является необязательной. Если переменные не инициализированы при объявлении, то применяются значения по умолчанию: для целочисленных типов данных — 0, для строковых — пустая строка.

1.11 Инструкции языка

Инструкции языка представляют из себя наборы команд и языка и позволяют пользователю использовать функционал языка программирования. В языке программирования PVR-2025 применяются инструкции, представленные в таблице 1.6.

Таблица 1.6 - Инструкции языка

Инструкция языка	Синтаксис
Вызов функций	<идентификатор функции>(<идентификатор / литерал>,…)
Возврат из функции	Ret <идентификатор> | <литерал> | <выражение>;
Объявление переменной	Let <тип данных> <идентификатор>;
Объявление переменной с инициализацией	Let <тип данных> <идентификатор> = <значение | литерал>;
Присваивание	<идентификатор>=<литерал>|<идентификатор><значение>;
Вывод данных	output <идентификатор> | <литерал>;
Объявление функции	<тип данных> Func <идентификатор> (<тип данных> <идентификатор>, …) {…}
Оператор ветвления	if (<условие>) { <инструкции> }
Оператор выбора	switch (<выражение>) { case <литерал>: <инструкции> break; … default: <инструкции> break; }

Использование инструкций, не указанных выше, является недопустимым.

1.12 Операции языка

Язык программирования PVR-2025 поддерживает операции, представленные в таблице 1.7 .

Таблица 1.7 - операции языка

Операции языка	Описание
Сложение +	Арифметический оператор. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Вычитание -	Арифметический оператор. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Умножение *	Арифметический оператор. Приоритетность: 2; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Деление нацело /	Арифметический оператор. Приоритетность: 2; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Остаток от деления %	Арифметический оператор. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Присвоение =	Приоритетность: 4; количество операндов: 2.
Изменение приоритета операции ( )	Приоритетность: 1; количество операндов: 1.
Конец инструкции ;	Приоритетность: 4.
Перечисление параметров ,	Приоритетность: 4.
Знак меньше <	Оператор сравнения. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.

Окончание таблицы 1.7

Знак больше >	Оператор сравнения. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Равенство ==	Оператор сравнения. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Неравенство !=	Оператор сравнения. Приоритетность: 3; количество операндов: 2; допустимые типы используемых операндов: целочисленный; допустимые типы результата: целочисленный.
Знак меньше/равно <=	Оператор сравнения. Приоритет: 3; количество операндов: 2; допустимые типы операндов: целочисленный; результат: целочисленный.
Знак больше/равно <=	Оператор сравнения. Приоритет: 3; количество операндов: 2; допустимые типы операндов: целочисленный; результат: целочисленный.

Во всех операциях, где количество операндов равно 2, первый и второй операнды должны быть одинакового типа данных. Операции с одинаковым приоритетом выполняются в порядке записи, слева направо. Использование операций отличных от объявленных в таблице не допускается.

1.13 Выражения и их вычисления

В языке PVR-2025 допускаются арифметические выражения с целочисленными данными. В них можно использовать все объявленные в языке операции. Операнды в математических выражениях должны быть целочисленными. Не допускается запись подряд двух операндов или двух арифметических операций. Вычисления подвыражений осуществляется в порядке приоритетности операций. Операции одного приоритета выполняются последовательно слева направо, что обеспечивает однозначный результат вычисления сложных выражений. Операндами выражения могут являться переменные, литералы и результаты вызова функций.

Примеры выражений: a + b; x * y.

1.14 Конструкции языка

Основные программные конструкции языка программирования, такие как фунции, основная программа, условный оператор, оператор выбора представлены в таблице 1.8. Эти конструкции описывают базовый управляющий каркас программы и определяют порядок выполнения вычислений и обработки данных. В рассмотренной задаче основная программа обращается к функции add3,

использует условные переходы if и оператор выбора switch для проверки значений переменных.

Таблица 1.8 - Программные конструкции языка

Конструкция	Представление в языке

Функция	Описание подпрограммы, имеющей имя, список параметров и тип возвращаемого значения. Объявляется ключевым словом Func после типа результата. В теле функции выполняются инструкции и оператор Ret возвращает значение:
	`int Func add3(int x, int y, int z) {`
	`Let int r;`
	`r = x + y;`
	`r = r + z;`
	`Ret r;`
	`}`

Основная программа	Точка входа в приложение. Обозначается ключевым словом Main, внутри фигурных скобок располагаются объявления переменных, вычисления и вывод результатов:
	Main {
	Let int a = 10;
	...
	output 'Done';
	}

Условный оператор	Конструкция ветвления, позволяющая выполнять блок инструкций при истинности логического выражения. Условие записывается в скобках после ключевого слова if, тело — в фигурных скобках:
	if (a == 10)
	{
	output 1;
	}

Оператор выбора	Конструкция многовариантного выбора по значению выражения. После ключевого слова switch указывается выражение, внутри фигурных скобок перечисляются метки case и ветвь default:
	switch (a)
	{
	case 10:
	output 100;
	break;
	default:
	output 110;
	break;
	}

1.15 Область видимости идентификаторов

Идентификаторы не могут объявляться вне функций. Внутри таких конструкций как цикл или функция, создается локальная переменная, область видимости которой ограничивается блоком кода, в котором она была объявлена. Не предусматривается определение пользовательских программных блоков.

1.16 Семантические проверки

Правила семантической проверки исходного текста языка PVR-2025:

- идентификаторы не могут повторяться в пределах области видимости;

- тип значения инициализации переменной должен совпадать с типом переменной;

- тип значения передаваемого в функцию параметра при вызове должен совпадать с типом параметра в объявлении функции;

- операнды в арифметических и логических выражениях должны быть одного типа;

- тип значения возвращаемого из функции должен совпадать с типом функции;

- не допускается переопределение типов данных;

- проверка соответствия литералов их диапазонам допустимых значений.

1.17 Распределение оперативной памяти на этапе выполнения

Скомпилированный код использует две области памяти: сегмент данных и сегмент кода. В сегмент данных заносятся переменные, литералы, параметры функций.

1.18 Стандартная библиотека и её состав

Это набор готовых функций, классов, модулей и других компонентов, которые входят в язык программирования по умолчанию и доступны для использования в любой программе, не требуя отдельной установки, чтобы упростить разработку общих задач, таких как ввод-вывод, работа со строками, математика или коллекции данных. В состав стандартной библиотеки входят функции, перечисленные и описанные в таблице 1.9.

Таблица 1.9 - Стандартная библиотека StdLibrary языка программирования PVR-2025

Функция	Описание	Параметры	Возвращаемое значение
IntToStr(x)	Преобразует целочисленный параметр в строку. Используется для получения текстового вида числа.	x – целочисленный параметр, значение переменной x	Строка, содержащая представление числа x.
StrLen(s)	Вычисляет длину строки. Применяется для определения количества символов в строковом параметре.	s – строковый параметр, результат вызова IntToStr	Целочисленная длина строки s.
output(value)	Выводит на консоль значение целочисленного или строкового параметра/литерала.	value – идентификатор переменной или строковый литерал ('Start', a, o3, 'Done')	Возвращаемое значение отсутствует.

Стандартная библиотека подключается автоматически при генерации кода на языке JavaScript. Применяется для использования в программном коде включенных в библиотеку функций.

1.19 Ввод и вывод данных

Ввод и вывод данных в языке программирования PVR-2025 осуществляется с помощью оператора output <идентификатор>|<литерал>. Оператор output применяется для вывода целочисленных и строковых значений на консоль.

1.20 Точка входа

Точкой входа в программе является ключевое слово “Main”. Точка входа не может отсутствовать или быть переопределена, это вызовет ошибку компилятора. Возвращаемое значение целочисленного типа.

1.21 Препроцессор

Препроцессор в языке программирования PVR-2025 не предусматривается.

1.22 Соглашения о вызовах

В языке программирования PVR-2025 соглашение stdcall, все параметры передаются в стек справа налево, память освобождается вызываемым кодом.

1.23 Объектный код

Языка программирования PVR-2025 транслируется в язык JavaScript.

1.24 Классификация сообщений транслятора

Транслятор генерирует сообщения об ошибках пользователю в процессе выполнения при их возникновении. Классификация сообщений об ошибках языка PVR-2025 представлена в таблице 1.10.

Таблица 1.10 – Классификация сообщений транслятора

Коды ошибок	Описание ошибок
0-99	Системные ошибки
100-199	Ошибки параметров
200-299	Ошибки лексического анализа
300-399	Ошибки синтаксического анализа
400-499	Ошибки семантического анализа
500-599	Ошибки генерации кода

В случае возникновения ошибки выполнение прерывается и информация об ошибке выводится в консоль и записывается в log-файл.

1.25 Контрольный пример

Контрольный пример на языке PVR-2025 представлен в приложении А.

2 Структура транслятора

2.1 Компоненты транслятора их назначение и принципы взаимодействия

Транслятор преобразует программу, написанную на языке PVR-2025 в программу на языке JavaScript. Компонентами транслятора являются лексический, синтаксический и семантический анализаторы, а также генератор кода на язык JavaScript. Принцип их взаимодействия представлен на рисунке 2.1

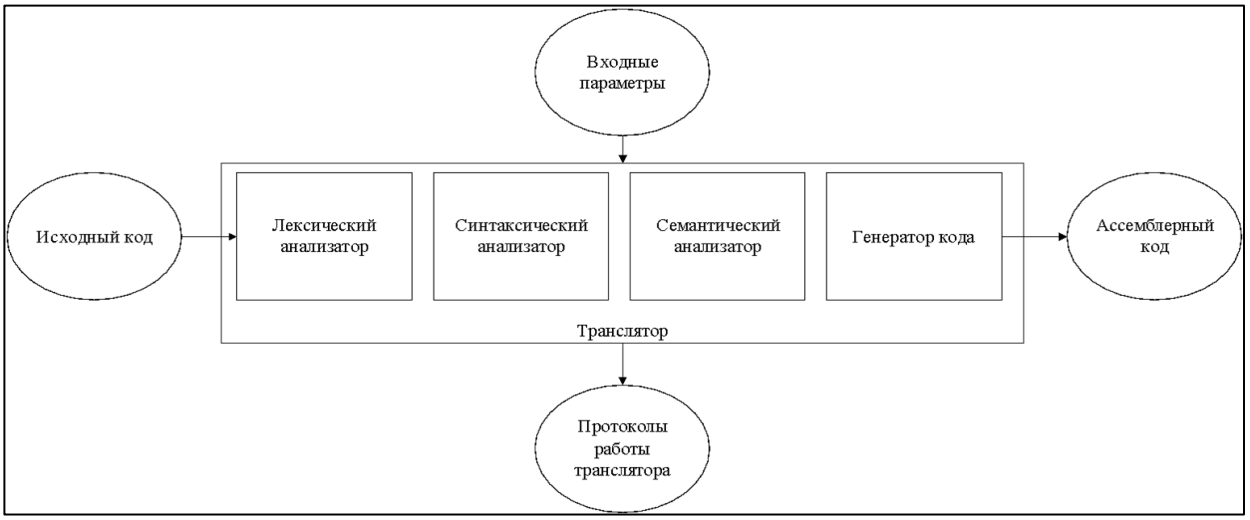


Рисунок 2.1 – Структура транслятора

Лексический анализ – первый этап трансляции. Назначением лексического анализатора является нахождение ошибок лексики языка и формирование таблицы лексем и таблицы идентификаторов.

Синтаксический анализ – второй этап, предназначенная для распознавания синтаксических конструкций. Входным параметром для синтаксического анализа является таблица лексем. Синтаксический анализатор распознаёт синтаксические конструкции, выявляет синтаксические ошибки при их наличии и формирует дерево разбора.

Семантический анализ — третий этап. На ней осуществляется проверка исходной программы на семантическую согласованность с определением языка, т.е. проверяется правильность текста исходной программы с точки зрения семантики.

Генератор кода – четвертый этап транслятора, выполняющий генерацию JavaScript кода на основе полученных данных на предыдущих этапах трансляции. Генератор кода принимает на вход таблицы идентификаторов и лексем и транслирует код на языке PVR-2025 в код на языке JavaScript.

2.2 Перечень параметров транслятора

Входные параметры представлены в таблице 2.1.

Таблица 2.1 – Входные параметры транслятора языка PVR-2025

Входной параметр	Описание	Значение по умолчанию
-in:<имя_файла>	Входной файл с расширением .txt, в котором содержится исходный код на PVR-2025. Это обязательный параметр.	<имя_in_файла>.txt
-log:<имя_файла>	Файл с расширение .log, определяет файлы, содержащие результат работы программы:	<имя_in_файла>.log
-out:<имя_файла>	Файл для записи результата работы транслятора	<имя_in_файла>.asm

В данной таблице были представлены входные параметры трансллтора PVR-2025, такие как -in, out,log.

2.3 Протоколы, формируемые транслятором

В процессе трансляции формируется файл протокола, в котором отображается: время выполнения трансляции, количество проанализированных строк и символов, ошибки в случае их присутствия, а также таблицы лексем и идентификаторов. Формируется файл asm.asm, в котором содержится код на языке JavaScript.

3 Разработка лексического анализатора

3.1 Структура лексического анализатора

Лексический анализатор является частью транслятора, которая выполняет лексический анализ. На вход лексического анализатора поступает обработанный и разбитый на отдельные компоненты исходный код на языке программирования. На выходе генерируются таблица лексем и таблица идентификаторов. Структура лексического анализатора представлена ниже на рисунке 3.1

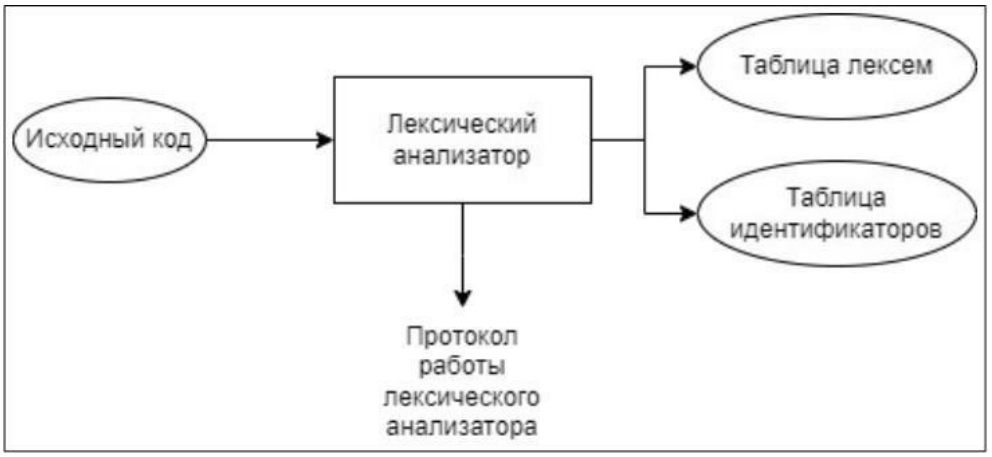


Рисунок 3.1 — Структура лексического анализатора

3.2 Входные и выходные данные лексического анализатора

Входными данными лексического анализатора является обработанный и разбитый на отдельные компоненты исходный код на языке программирования PVR-2025. Выходными данными лексического анализатора являются таблица лексем и таблица идентификаторов. Таблицы лексем и идентификаторов будут использованы на следующих этапах компилятора.

3.3 Параметры лексического анализатора

Параметрами лексического анализатора являются обработанный и разбитый на отдельные компоненты исходный код, таблицы лексем и идентификаторов.

3.4 Алгоритм лексического анализа

Алгоритм работы лексического анализатора выглядит следующим образом:

- Лексический анализатор распознаёт и разбирает цепочки исходного текста программы, удаляет лишние пробелы;

- для выделенной части входного потока выполняется функция распознавания лексемы;

- при успешном распознавании информация о выделенной лексеме заносится в таблицу лексем и таблицу идентификаторов, и алгоритм возвращается к первому этапу;

- при неуспешном распознавании выдается сообщение об ошибке.

3.5 Контроль входных символов

Перед трансляцией исходный код на языке программирования PVR-2025 проходит проверку на допустимость символов. Из входного файла считывается по одному символу и каждый символ проверяется на то, является ли он разрешенным.

Таблица для контроля входных символов основана на таблице Windows-1251, представлена в рисунке 3.3.

```
int code[256] = {
    F, F, F, F, F, F, F, F, F, I, BR, F, F, BR, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    I, T, L, T, T, T, T, T, T, T, T, T, T, T, T, T,
    T, T, T, T, T, T, T, T, T, T, T, T, T, T, T, T,
    T, T, T, T, T, T, T, T, T, T, T, T, T, T, T, T,
    T, T, T, T, T, T, T, T, T, T, T, T, T, T, T, T,
    T, T, T, T, T, T, T, T, T, T, T, T, T, T, T, T,
    T, T, T, T, T, T, T, T, T, T, T, T, T, T, T, F,

    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F,
    F, F, F, F, F, F, F, F, F, F, F, F, F, F, F, F
};
```

Рисунок 3.3 — Таблица контроля входных символов

Происходит проверка каждого символа на соответствие его значения значению в таблице Windows-1251. Применяемые в таблице символы: T – разрешённый символ, F – запрещённый символ, I – игнорируемый символ.

3.6 Удаление избыточных символов

Избыточными символами являются пробельные символы и символы табуляции.

Алгоритм удаления избыточных символов:

- Файл с исходным кодом программы посимвольно считывается;

- При встрече символа пробела или символа знака табуляции, в отличии от других символов-сепараторов, не записываем их в таблицу лексем;

- проверяем с помощью цикла: если следующий символ в потоке будет знаком табуляции или пробела – пропускаем этот символ до тех пор, пока следующим символом будет не \t и не ‘ ‘ .

3.7 Перечень ключевых слов

Для создания промежуточного представления исходной программы лексический анализатор преобразует исходный текст, заменяя лексические единицы на лексемы. Перечень ключевых слов языка, сепараторов, символов операций, а также соответствующих им лексем и регулярных выражений, представлен в таблицу 3.1

Таблица 3.1 – Соответствие ключевых слов, символов операций и сепараторов с лексемами

Токен	Лексема	Пояснение
Main	m	Главная функция
Let	e	Объявление переменной
Func	f	Объявление функции
output	o	Вывод данных на консоль
int, string	t	Названия типов данных языка
Идентификатор	i	Идентификатор переменной, функции либо параметра функции
Литерал	l	Литерал любого доступного типа
Ret	r	Выход из функции/процедуры
IntToStr, StrLen	s	Функции стандартной библиотеки
if	q	Начало условного оператора
switch, case, default	w	Оператор выбора
;	;	Разделение выражений
,	,	Разделение параметров функций
>,>=,<,>=,!=,==	>,>=,<,>=,!=,==	Знаки операций сравнения
+,-,*,/,%	p	Арифметические операции
},{	},{	Закрытие/открытие блока функции
(	(	Передача параметров в функцию
)	)	Закрытие блока для передачи параметров, приоритет операций

В данной таблице представлено соответствие ключевых слов, символов операций и сепараторов с лексемами.

3.8 Основные структуры данных

Основными структурами данных лексического анализатора являются таблица лексем и таблица идентификаторов.

Таблица лексем состоит из лексем, полученной при разборе исходного текста программы на языке PVR-2025, номера строки в исходном коде и служебной информации о типе лексемы и приоритете операции. К лексемам относятся ключевые слова (Main, Let, Func, output, операторы if, switch), имена переменных и функций (add3, a, b, c, x, len, o1, o2, o3, s), а также числовые и строковые литералы (10, 3, 5, 'Start ', 'Done' и др.). Если лексема является идентификатором, в таблице лексем дополнительно хранится индекс записи в таблице идентификаторов, где заданы тип данных и дополнительные характеристики идентификатора.

```
struct Entry { char lexema;
          int line;
          int idxTI;
          int priority;
          operations op; };
     struct LexTable {
          int maxsize;
          int size;
          Entry* table; };
```

Листинг 3.1 - Структура таблицы лексем

Таблица идентификаторов содержит имя идентификатора, номер в таблице лексем, тип данных, тип идентификатора и значение (или параметры функций). Структура таблицы идентификаторов представлен в листинге 3.4.

```
struct Entry { int idxFirstLE;
          char visibility[ID_MAXSIZE];
          char id[ID_MAXSIZE];
          IDDATATYPE idDataType;
          IDTYPE idtype;
          int countOfPar = 0;
          union {   int vint = 0;
               struct {
```

```
                int len;
                char str[TI_STR_MAXSIZE - 1]; } vstr;
        } value;};
struct IdTable {
        int maxsize;
        int size;
        Entry* table;  };.
```

Листинг 3.2 - Структура таблицы идентификаторов

3.9 Структура и перечень сообщений лексического анализатора

В структуре сообщений содержится информация о номере ошибки, префикс, пояснительный текст сообщения об ошибке, номер строки и номер позиции исходного текста программы, где было вызвано сообщение. При возникновении сообщения дальнейшая обработка лексическим анализатором прекращается. Перечень сообщений представлен в таблице 3.2.

Таблица 3.2 — Перечень сообщений лексического анализатора

Номер сообщения	Текст сообщения
200	Таблица лексем переполнена
201	Нераспознанная лексема
202	Таблица идентификаторов переполнена
203	Перезапись идентификатора
204	Выход за границу таблицы лексем (или идентификаторов)
205	Не удалось создать файл с лексемами (или идентификаторами)
206	Слишком длинный литерал
207	Неверный формат строкового литерала
208	Слишком длинный идентификатор
209	Неизвестная переменная
210	Слишком большой целочисленный литерал

3.10 Принцип обработки ошибок

При возникновении ошибки работа лексического анализатора прекращается. Для обработки ошибок используется таблица с сообщениями. Возникающие ошибки фиксируются в протокол заданный входными параметрами.

3.11 Контрольный пример

Результат работы лексического анализатора в приложении Б.

4 Разработка синтаксического анализатора

4.1 Структура синтаксического анализатора

Синтаксический анализатор выполняет синтаксический анализ, то есть проверяет исходный корд программы на соответствие правилам грамматики. На вход синтаксического анализатора поступают полученные ранее таблица идентификаторов и таблица лексем, на выход дерево разбора. На рисунке 4.1 представлена структура синтаксического анализатора.

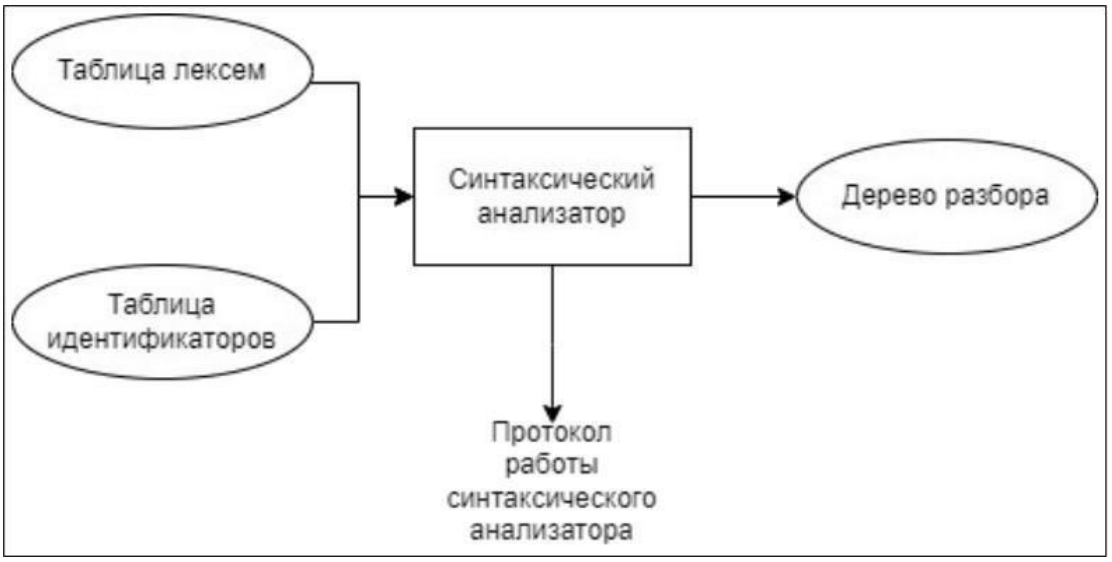


Рисунок 4.1 – Структура синтаксического анализатора.

На этом рисунке представлена работа синтаксического анализатора, который является вторым этапом в работе транслятора после лексического анализатора. На рисунке явно указана схема, на которой представлены входные и выходные данные синтаксического анализатора.

4.2 Контекстно-свободная грамматика, описывающая синтаксис языка

В синтаксическом анализаторе для языка PVR-2025 используется контекстно-свободная грамматика $G=<T, N, P, S>$, где:

T – множество терминальных символов (было описано в главе 1),

N – множество нетерминальных символов,

P – множество правил языка,

S – начальный символ грамматики, являющийся нетерминалом.

Грамматика языка программирования PVR-2025 представлена в приложении B. TS – это терминальные символы, которыми являются сепараторы, знаки арифметических операций и некоторые строчные буквы. NS – это нетерминальные символы, представленные несколькими заглавными буквами латинского алфавита.

Перечень составляющих грамматику языка правил и описание нетерминальных символов представлены в приложении Г.

4.3 Построение конечного магазинного автомата

Конечный автомат с магазинной памятью представляет собой семерку $M = \langle Q, V, Z, \delta, q_0, z_0, F \rangle$. Подробное описание компонентов магазинного автомата представлено в таблице 4.1.

Таблица 4.1 – Описание компонентов магазинного автомата

Компонента	Определение	Описание
Q	Множество состояний автомата	Состояние автомата представляет из себя структуру, содержащую позицию на входной ленте, номера текущего правила и цепочки и стек автомата
V	Алфавит входных символов	Алфавит представляет из себя множества терминальных и нетерминальных символов, описание которых содержится в таблица 3.2 и 4.1 соответственно
Z	Алфавит специальных магазинных символов	Алфавит магазинных символов содержит стартовый символ и маркер дна стека (представляет из себя символ $)
δ	Функция переходов автомата	Функция представляет из себя множество правил грамматики, описанных в таблице 4.1
q_0	Начальное состояние автомата	Состояние, которое приобретает автомат в начале своей работы. Представляется в виде стартового правила грамматики
z_0	Начальное состояние магазина автомата	Символ маркера дна стека $
F	Множество конечных состояний	Конечные состояние заставляют автомат прекратить свою работу. Конечным состоянием является пустой магазин автомата и совпадение позиции на входной ленте автомата с размером ленты

4.4 Основные структуры данных

Основные структуры данных синтаксического анализатора включают в себя структуру магазинного автомата для разбора исходной ленты и структуру грамматики Грейбах для описания правила языка PVR-2025. Данные структуры представлены в приложении Д.

4.5 Описание алгоритма синтаксического разбора

Алгоритм синтаксического разбора:

1. Стартовый символ записывается в магазин.

2. На основе ранее полученных таблиц формируется входная лента.

3. Запускается автомат.

4. В соответствии с нетерминальным символом выбирается цепочка и записывается в магазин в обратном порядке.

5. При совпадении терминалов в стеке и ленте данный терминал удаляется из ленты и стека, в противном случае происходит возвращение в предыдущее сохраненное состояние и выбирается другая цепочка нетерминала.

6. При встрече нетерминала в магазине переходим к пункту 4.

7. Если символ достиг дна стека, а лента в этот момент пустая, синтаксический анализ выполнен успешно. В противном случае генерируется исключение. После 3 исключений работа синтаксического анализатора завершается.

4.6 Параметры синтаксического анализатора

Входным параметром для синтаксического анализатора является таблица лексем, полученная на предыдущем этапе, и правила контекстно-свободной грамматики в форме Грейбаха. Выходным параметром синтаксического анализатора является дерево разбора.

4.7 Структура и перечень сообщений синтаксического анализатора

В таблице 4.2 представлен перечень сообщений синтаксического анализатора.

Таблица 4.2 – Перечень сообщений синтаксического анализатора

Номер сообщения	Текст сообщения
300	Неверная структура программы
301	Ошибочный оператор
302	Ошибка в выражении
303	Ошибка в параметрах функции
304	Ошибка в параметрах вызываемой функции
305	Ошибка в условии цикла

В таблице явно представлен перечень сообщений синтаксического анализатора.

4.8 Принцип обработки ошибок

Принцип обработки ошибок в синтаксическом анализаторе следующий:

1. Синтаксический анализатор перебирает все правила и цепочки правил грамматики для нахождения подходящего соответствия с конструкцией, представленной в таблице лексем.

2. При невозможности подобрать подходящую цепочку, генерируется соответствующая ошибка и записывается в протокол работы, программа останавливается.

3. В случае возникновения ошибки выведется диагностическое сообщение после вывода трассировки синтаксического анализатора. Если возникает более 3-х ошибок, работа синтаксического анализатора завершается.

4.9 Контрольный пример

Результатом работы синтаксического анализатора является дерево разбора. Трассировка синтаксического анализатора и дерево разбора представлено в приложении Е

5 Разработка семантического анализатора

5.1 Структура семантического анализатора

Семантический анализатор использует синтаксическое дерево и информацию из таблицы идентификаторов для смысловой проверки исходного текста на соответствие семантическим правилам. Он дополняет внутреннее представление программы операторами и действиями, неявно предусмотренными семантикой исходного языка.

Семантический анализатор принимает на вход таблицу лексем, таблицу идентификаторов, дерево разбора. Далее осуществляются проверки на наличие ошибок.

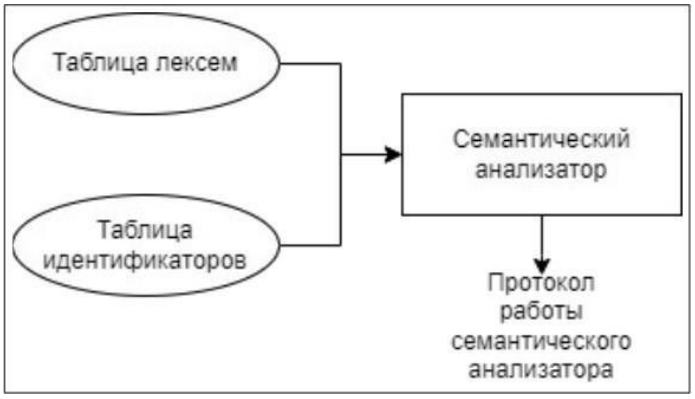


Рисунок 5.1 — Структура семантического анализатора

Семантический анализатор использует синтаксическое дерево и информацию из таблицы идентификаторов для смысловой проверки исходного текста на соответствие семантическим правилам.

5.2 Функции семантического анализатора

Семантический анализатор выполняет проверку на семантические правила языка программирования. За семантический анализ отвечает функция Analyze. Её входными параметрами является таблица лексем и таблица идентификаторов. Через цикл перебираются поочередно поступившие на вход лексемы и проверяются на соответствие существующим правилам.

5.3 Структура и перечень сообщений семантического анализатора

В таблице 5.1 представлен перечень сообщений семантического анализатора.

Таблица 5.1 – Сообщения семантического анализатора

Номер сообщения	Текст сообщения
400	Ret не может содержать функцию
401	Ошибка в аргументах вызываемой функции
402	Нельзя присвоить значение данного типа
403	Недопустимый тип аргумента цикла
404	Отсутствует точка входа
405	Переопределение переменной
406	Слишком большое количество функций
407	Функция Main не может использоваться более 1 раза

Структура сообщения семантического анализатора состоит из номера сообщения и текста сообщения.

5.4 Принцип обработки ошибок

При появлении ошибки работа семантического анализатора приостанавливается. Происходит запись ошибок в протокол с указанием номера ошибки и текстом сообщения ошибки.

5.5 Контрольный пример

Демонстрации ошибок, диагностируемых семантическим анализатором на разных этапах трансляции приведена в главе 8

6 Вычисление выражений

6.1 Выражения, допускаемые языком

В языке программирования PVR-2025 допускаемыми для вычисления выражениями являются выражения целочисленного типа данных, допускается вызов функций внутри выражений. Приоритет операций представлен в таблице 6.1.

Таблица 6.1 – Приоритеты операций

Операция	Значение приоритета
+	1
-	1
*	2
/	2
%	2
==, !=	3
<=, >=	3
)	0
(	0

В языке программирования присутствует приоритет операций, который был представлен в таблице выше.

6.2 Польская запись и принцип ее построения

Обратная польская запись это способ записи арифметических выражений, в которой операторы расположены после своих операндов. Достоинством данной записи является то, что представленные в ней выражения вычисляются за линейное время.

Выражения в языке программирования преобразовываются к обратной польской записи.

Алгоритм построения польской записи:

- исходная строка: выражение;

- результирующая строка: польская запись;

- стек: пустой;

- исходная строка просматривается слева направо;

- операнды переносятся в результирующую строку;

- операция записывается в стек, если стек пуст;

- операция выталкивает все операции с большим или равным приоритетом в результирующую строку;

- отрывающая скобка помещается в стек;

- закрывающая скобка выталкивает все операции до открывающей скобки, после чего обе скобки уничтожаются.

6.3 Программная реализация обработки выражений

Программная реализация алгоритма преобразования выражений в обратную польскую нотацию основана на функции startPolish, которая принимает как параметр адрес таблицы лексем и содержит цикл, в ходе которого перебираются все лексемы исходного кода. Далее преобразование к польской нотации осуществляется с помощью функции polishNotation, на вход которой поступают адрес таблицы лексем и полученное ранее количество лексем, происходит обработка данных в соответствии с описанным выше алгоритмом.

6.4 Контрольный пример

В листинге 6.1 представлена таблица лексем после преобразования к польской нотации контрольного примера исходного кода.

```
          Польская нотация
0001| tfi(ti,ti,ti){lti;
0002| i=iip;
0003| i=iip;
0004| ri;
0005| }m{lti=k;
0006| lti;
0007| lti;
0008| lti;
0009| lti;
0010| lti;
0011| i=k;
0012| i=i@1#;
0013| i=i@1#;
0014| i=k;
0015| i=iik@3###;
0016| ok;
0017| oi;
0018| ok;
0020| ok;
0021| oi;
0022| ok;
0023| lti;
0024| lti;
0025| lti;
0026| i=k;
0027| i=k;
0028| i=iip;
0029| ok;
0030| oi;
0031| ok;
0032| oi;
0033| ok;
0034| oi;
0035| ok;
0036| ok;
```

```
0037| x(ipk){ok;
0038| }ok;
0039| x(ipk){ok;
0040| }ok;
0041| x(ipi){ok;
0042| }ok;
0043| x(ipi){ok;
0044| }ok;
0045| ok;
0046|
s(i){ck:ok;
0047| b;
0048| d:ok;
0049| b;
0050| }ok;
0051| ok;
0052| }
```

Листинг 6.1 — Таблица лексем после преобразования к польской нотации

7 Генерация кода

7.1 Структура генератора кода

Генерация кода для языка PVR-2025 выполняется на JavaScript и является завершающей стадией трансляции. На вход генератора поступают таблица лексем и таблица идентификаторов, полученные после работы лексического анализатора. Структура генератора кода представлена на рисунке 7.1.

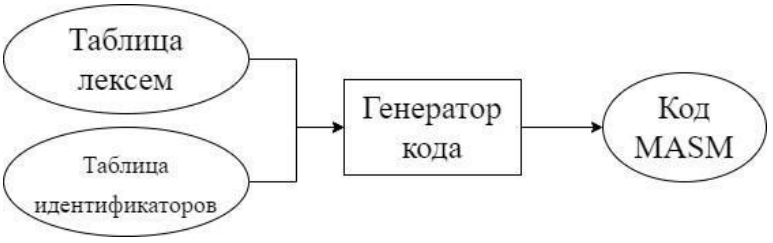


Рисунок 7.1 Структура генератора кода

Выходным результатом является файл с исходным кодом на JavaScript, формируемый в соответствии с последовательностью лексем (таблицей лексем). При возникновении ошибок генератор прекращает

выполнение, и выходной JavaScript-код не создаётся.

7.2 Представление типов данных в оперативной памяти

Модель хранения данных в PVR-2025 при генерации кода на JavaScript строится иначе, чем в ассемблере: отдельные сегменты памяти .data и .const не используются, а значения формируются как JavaScript-объекты и литералы. Идентификаторы представляются переменными/ссылками JavaScript, а литералы — как встроенные значения (числа, строки и т.д.), которые напрямую вставляются в генерируемый код.

Соответствие типов PVR-2025 типам JavaScript задаётся на уровне генератора: для каждого идентификатора из таблицы идентификаторов выбирается подходящий JS-тип и способ инициализации, после чего создаётся корректный фрагмент выходного .js файла.

Пример соответствия типов (под JS):

1) int → number (целочисленное значение хранится как число JavaScript).

2) string → string (строка хранится как строка JavaScript, без требований про нуль-терминатор).

7.3 Статическая библиотека

В языке программирования PVR-2025 предусматривается статическая библиотека. В ней содержатся описанные на языке C++ функции.

Список функций и их подробное описание было представлено в главе 1.

В коде JavaScrip автоматически генерируется объявление функций статической библиотеки. Стандартная библиотека подключается автоматически.

7.4 Особенности алгоритма генерации кода

Генерация кода для PVR-2025 выполняется на языке JavaScript и является заключительным этапом трансляции. На вход модулю генерации передаются таблица лексем и таблица идентификаторов, сформированные на предыдущих этапах.

В процессе работы генератор последовательно обрабатывает элементы таблицы лексем, определяет контекст (объявление, присваивание, вызов функции, оператор switch, вывод и т.д.) и формирует соответствующие фрагменты JavaScript-кода. Для идентификаторов (переменных и функций) используются сведения из таблицы идентификаторов: имя, тип, область видимости/параметры и другая необходимая информация.ЫІ

Результатом является файл с исходным кодом на JavaScript (обычно с расширением .js), который формируется на основе таблицы лексем и данных таблицы идентификаторов. При возникновении ошибок генератор прекращает работу и выходной файл (или его корректная версия) не формируется.

7.5 Параметры, управляющие генерацией кода

На вход генератора кода передаются таблица идентификаторов и таблица лексем. На основании этих данных формируется выходной файл с исходным кодом на JavaScript (с расширением .js).

7.6 Контрольный пример

Результат генерации JavaScript кода на основе контрольного примера из приложения А приведен в приложении Ж.

8. Тестирование транслятора

8.1 Общие положения

При возникновении ошибки на каком-либо этапе трансляции, она обрабатывается в главном файле программы: ошибка выводится на консоль и записывается в файл логирования с расширением .log. Если ошибок обнаружено не было, будет выведен в консоль текст об успешной компиляции файла

8.2 Результаты тестирования

Проверка на первом этапе трансляции подразумевает проверку на допустимость символов. В языке программирования PVR-2025 не допускается использовать запрещённые символы, запрещенные входным алфавитом. При использовании в исходном тексте программы генерируется ошибка, представленная в таблице 8.1.

Таблица 8.1 – Тестирование проверки на допустимость символов

Исходный код	Диагностическое сообщение
Let пер №	Ошибка 100: Недопустимый символ в исходном коде (-in), строка 1

На этапе лексического анализа проверяется соответствие единиц языка ключевым словам, правилам объявления переменным, правилам записи идентификаторов и так далее. Результаты тестирования лексического анализатора показаны в таблице 8.2.

Таблица 8.2 – Тестирование лексического анализатора

Исходный код	Диагностическое сообщение
Lett int x1;	Ошибка 201: Нераспознанная лексема, строка 15
Let int x; Let int x;	Ошибка 203: Перезапись идентификатора, строка 16
Let int xxxxxxxxxxxxxxxxxx;	Ошибка 208: Слишком длинный идентификатор, строка 15

Main
{
x=10;
…
}
	Ошибка 209: Неизвестная переменная, строка 22

На этапе синтаксического анализа проверяется правильность синтаксических

конструкций языка. Результаты тестирования синтаксического анализатора

показаны в таблице 8.3.

Таблица 8.3 – Тестирование синтаксического анализатора

Исходный код	Диагностическое сообщение
Main)	Ошибка 300: Неверная структура программы (лишняя)).
int Func add3(int x, int y {	Ошибка 301: Отсутствует закрывающая скобка) в заголовке функции.
x = ;	Ошибка 302: Ошибка в выражении, отсутствует правый операнд присваивания.
o3 = o1 + ;	Ошибка 302: Ошибка в выражении, отсутствует второй операнд сложения.
if (a = 10)	Ошибка 303: Использован оператор присваивания вместо операции сравнения в условии.
switch a) {	Ошибка 304: Отсутствуют круглые скобки вокруг выражения оператора выбора.
case : output 100;	Ошибка 305: Отсутствует константа после ключевого слова case.

На этапе семантического анализа проверяется соответствие семантическим правилам языка. Итоги тестирования семантического анализатора на корректное обнаружение семантических ошибок приведены в таблице 8.4.

Таблица 8.4 – Тестирование семантического анализатора

Исходный код	Диагностическое сообщение
Let string sa = ' text1'; Let string sb = 10; Let string sc; sc = strf(sa, sb); output sc;	Ошибка 312: Семантическая ошибка: Несоответствие типов данных Строка 29 позиция -1

Окончание таблицы 8.4

string Func strf(string a, string b)
{
Let int cn;
cn = a;
Ret cn;
}
	Ошибка 314: Семантическая ошибка: Функция возвращает неверный тип данных
	Строка 11 позиция -1
	Ошибка 312: Семантическая ошибка: Несоответствие типов данных
	Строка 10 позиция -1

if ('a' < 'f')
{
output ;
}
	Ошибка 403: Недопустимый тип аргумента цикла, строка 17

Let int a; Let int a;	Ошибка 405: Переопределение переменной, строка 2
Main{…} Main{…}	Ошибка 407: Функция main не может использоваться более 1 раза, строка 15

Итоги тестирования семантического анализатора на корректное обнаружение семантических ошибок.

Заключение

В ходе выполнения курсовой работы был разработан транслятор и генератор кода для языка программирования PVR-2025 со всеми необходимыми компонентами. Таким образом, были выполнены основные задачи данной курсовой работы:

- разработана и сформулирована спецификация языка PVR-2025;
- разработаны конечные автоматы и алгоритмы на их основе для осуществления лексического анализа;
- осуществлена программная реализация лексического анализатора, распознающего допустимые цепочки языка программирования PVR-2025;
- разработана и приведена к нормальной форме Грейбах, контекстно-свободная грамматика для описания синтаксически верных конструкций языка;
- осуществлена программная реализация синтаксического анализатора;
- разработан семантический анализатор, осуществляющий проверку используемых инструкций на соответствие семантическим правилам языка;
- разработан транслятор кода на язык JavaScript;
- проведено тестирование всех вышеперечисленных компонентов.

Окончательная версия языка PVR-2025 включает:

- 2 типа данных — целочисленный (шестнадцатеричное и двоичное) и строковый;
- наличие функции вывода текста, стандартных функций из библиотеки языка;
- возможность определения пользовательских функций и их вызова, вызова функций стандартной библиотеки;
- наличие арифметических операторов для вычисления выражений, операторы сравнения;
- наличие условного оператора switch;
- структурированная и классифицированная система для обработки ошибок пользователя.

Данная курсовая работа позволила получить необходимое представление о структурах и процессах, использующихся при построении трансляторов, структуре языков программирования.

Список использованных источников

1. Курс лекций по предмету «Конструирование программного обеспечения» Наркевич А.С.

2. Страуструп, Б. Принципы и практика использования C++ / Б. Страуструп – 2009 – 1238 с.

3. Прата, С. Язык программирования C++. Лекции и упражнения / С. Прата. – М., 2006 — 1104 с.

4. Герберт, Ш. Справочник программиста по C/C++ / Шилдт Герберт. – 3-е изд. – Москва: Вильямс, 2003. - 429 с.

5. Ахо, А. Компиляторы: принципы, технологии и инструменты / А. Ахо, Р. Сети, Дж. Ульман. – М.: Вильямс, 2003. – 768с.

Приложение А

```
short main()
{
    short a = 10;
    short b = 0h0A;
    short c = 0b1010;

    string s1 = "apple";
    string s2 = "banana";
    string s3 = "apple";

    short eq_ab = (a == b);
    short ne_bc = (b != c);
    short lt_ab = (a < b);
    short gt_bc = (b > c);
    short le_ac = (a <= c);
    short ge_cb = (c >= b);

    output eq_ab;
    output ne_bc;
    output lt_ab;
    output gt_bc;
    output le_ac;
    output ge_cb;

    short cmp12 = strcmp(s1, s2);
    short cmp13 = strcmp(s1, s3);
    short cmp21 = strcmp(s2, s1);

    output cmp12;
    output cmp13;
    output cmp21;

    short len1 = strlen(s1);
    short len2 = strlen(s2);
    short len3 = strlen(s3);

    output len1;
    output len2;
    output len3;

    short code = 2;

    switch (code)
    {
        case 0:
```

```
            output "case 0";
            break;

        case 1:
            output "case 1";
            break;

        case 2:
            output "case 2";
            break;

        case 0h03:
            output "case 3";
            break;

        default:
            output "default";
            break;
    }

    return 0;
}
```

Листинг А.1 - Исходный код на языке PVR-2025

Приложение Б

```
struct Entry {
          char lexema;
          int line;
          int idxTI;
          int priority;
          operations op;
     };
struct LexTable {
          int maxsize;
          int size;
          Entry* table;
     };
```

Листинг Б.1 – Реализация таблицы лексем

```
namespace IT {
     enum IDDATATYPE { INT = 1, STR = 2, PROC = 4 };
     enum IDTYPE { V = 1, F = 2, P = 3, L = 4, OP = 5 };

     struct Entry {
          int idxFirstLE;
          char visibility[ID_MAXSIZE];
          char id[ID_MAXSIZE];

          IDDATATYPE idDataType;
          IDTYPE idtype;
          int countOfPar = 0;
          union {
               int vint = 0;
               struct {
                    int len;
                    char str[TI_STR_MAXSIZE - 1];
               } vstr;
          } value;
     };

     struct IdTable {
          int maxsize;
          int size;
          Entry* table;
     };

     IdTable Create(int size);
     void Add(IdTable& idtable, Entry entry);
     Entry GetEntry(IdTable& idtable, int n);
     int IsId(IdTable& idtable, char id[ID_MAXSIZE]);

     void Delete(IdTable& idtable);
     void ShowTable(IdTable& idtable, std::ostream& stream_out);
     void PrintTable(IdTable& idtable);
}
```

Листинг Б.1 – Реализация таблицы идентификаторов

Приложение В

```
===== LEX TABLE (TOKENS, BEFORE RPN) =====
#     line  lex   name             idxTI   prio  op
1     1     t     LEX_SHORT        -1      0     -
2     1     m     LEX_MAIN         0       0     -
3     1     (     LEX_LEFTTHESIS   -1      0     -
4     1     )     LEX_RIGHTTHESIS  -1      0     -
5     3     {     LEX_LEFTBRACE    -1      0     -
6     5     t     LEX_SHORT        -1      0     -
7     5     i     LEX_ID           1       0     -
8     5     =     LEX_ASSIGN       -1      0     -
9     5     l     LEX_LITERAL      2       0     -
10    5     ;     LEX_SEMICOLON    -1      0     -
11    7     t     LEX_SHORT        -1      0     -
12    7     i     LEX_ID           3       0     -
13    7     =     LEX_ASSIGN       -1      0     -
14    7     l     LEX_LITERAL      2       0     -
15    7     ;     LEX_SEMICOLON    -1      0     -
16    9     t     LEX_SHORT        -1      0     -
17    9     i     LEX_ID           4       0     -
18    9     =     LEX_ASSIGN       -1      0     -
19    9     l     LEX_LITERAL      2       0     -
20    9     ;     LEX_SEMICOLON    -1      0     -
21    13    g     LEX_STRING       -1      0     -
22    13    i     LEX_ID           5       0     -
23    13    =     LEX_ASSIGN       -1      0     -
24    13    l     LEX_LITERAL      6       0     -
25    13    ;     LEX_SEMICOLON    -1      0     -
26    15    g     LEX_STRING       -1      0     -
27    15    i     LEX_ID           7       0     -
28    15    =     LEX_ASSIGN       -1      0     -
29    15    l     LEX_LITERAL      8       0     -
30    15    ;     LEX_SEMICOLON    -1      0     -
31    17    g     LEX_STRING       -1      0     -
32    17    i     LEX_ID           9       0     -
33    17    =     LEX_ASSIGN       -1      0     -
34    17    l     LEX_LITERAL      6       0     -
35    17    ;     LEX_SEMICOLON    -1      0     -
36    21    t     LEX_SHORT        -1      0     -
37    21    i     LEX_ID           10      0     -
38    21    =     LEX_ASSIGN       -1      0     -
39    21    (     LEX_LEFTTHESIS   -1      0     -
40    21    i     LEX_ID           1       0     -
41    21    p     LEX_OPERATOR     11      1     ==
42    21    i     LEX_ID           3       0     -
43    21    )     LEX_RIGHTTHESIS  -1      0     -
44    21    ;     LEX_SEMICOLON    -1      0     -
```

45	23	t	LEX_SHORT	-1	0	-
46	23	i	LEX_ID	12	0	-
47	23	=	LEX_ASSIGN	-1	0	-
48	23	(	LEX_LEFTTHESIS	-1	0	-
49	23	i	LEX_ID	3	0	-
50	23	p	LEX_OPERATOR	13	1	!=
51	23	i	LEX_ID	4	0	-
52	23	)	LEX_RIGHTTHESIS	-1	0	-
53	23	;	LEX_SEMICOLON	-1	0	-
54	25	t	LEX_SHORT	-1	0	-
55	25	i	LEX_ID	14	0	-
56	25	=	LEX_ASSIGN	-1	0	-
57	25	(	LEX_LEFTTHESIS	-1	0	-
58	25	i	LEX_ID	1	0	-
59	25	p	LEX_OPERATOR	15	1	<
60	25	i	LEX_ID	3	0	-
61	25	)	LEX_RIGHTTHESIS	-1	0	-
62	25	;	LEX_SEMICOLON	-1	0	-
63	27	t	LEX_SHORT	-1	0	-
64	27	i	LEX_ID	16	0	-
65	27	=	LEX_ASSIGN	-1	0	-
66	27	(	LEX_LEFTTHESIS	-1	0	-
67	27	i	LEX_ID	3	0	-
68	27	p	LEX_OPERATOR	17	1	>
69	27	i	LEX_ID	4	0	-
70	27	)	LEX_RIGHTTHESIS	-1	0	-
71	27	;	LEX_SEMICOLON	-1	0	-
72	29	t	LEX_SHORT	-1	0	-
73	29	i	LEX_ID	18	0	-
74	29	=	LEX_ASSIGN	-1	0	-
75	29	(	LEX_LEFTTHESIS	-1	0	-
76	29	i	LEX_ID	1	0	-
77	29	p	LEX_OPERATOR	19	1	<=
78	29	i	LEX_ID	4	0	-
79	29	)	LEX_RIGHTTHESIS	-1	0	-
80	29	;	LEX_SEMICOLON	-1	0	-
81	31	t	LEX_SHORT	-1	0	-
82	31	i	LEX_ID	20	0	-
83	31	=	LEX_ASSIGN	-1	0	-
84	31	(	LEX_LEFTTHESIS	-1	0	-
85	31	i	LEX_ID	4	0	-
86	31	p	LEX_OPERATOR	21	1	>=
87	31	i	LEX_ID	3	0	-
88	31	)	LEX_RIGHTTHESIS	-1	0	-
89	31	;	LEX_SEMICOLON	-1	0	-
90	35	o	LEX_OUTPUT	-1	0	-
91	35	i	LEX_ID	10	0	-

92	35	;	LEX_SEMICOLON	-1	0	-
93	37	o	LEX_OUTPUT	-1	0	-
94	37	i	LEX_ID	12	0	-
95	37	;	LEX_SEMICOLON	-1	0	-
96	39	o	LEX_OUTPUT	-1	0	-
97	39	i	LEX_ID	14	0	-
98	39	;	LEX_SEMICOLON	-1	0	-
99	41	o	LEX_OUTPUT	-1	0	-
100	41	i	LEX_ID	16	0	-
101	41	;	LEX_SEMICOLON	-1	0	-
102	43	o	LEX_OUTPUT	-1	0	-
103	43	i	LEX_ID	18	0	-
104	43	;	LEX_SEMICOLON	-1	0	-
105	45	o	LEX_OUTPUT	-1	0	-
106	45	i	LEX_ID	20	0	-
107	45	;	LEX_SEMICOLON	-1	0	-
108	49	t	LEX_SHORT	-1	0	-
109	49	i	LEX_ID	22	0	-
110	49	=	LEX_ASSIGN	-1	0	-
111	49	i	LEX_ID	23	0	-
112	49	(	LEX_LEFTTHESIS	-1	0	-
113	49	i	LEX_ID	5	0	-
114	49	,	LEX_COMMA	-1	0	-
115	49	i	LEX_ID	7	0	-
116	49	)	LEX_RIGHTTHESIS	-1	0	-
117	49	;	LEX_SEMICOLON	-1	0	-
118	51	t	LEX_SHORT	-1	0	-
119	51	i	LEX_ID	24	0	-
120	51	=	LEX_ASSIGN	-1	0	-
121	51	i	LEX_ID	23	0	-
122	51	(	LEX_LEFTTHESIS	-1	0	-
123	51	i	LEX_ID	5	0	-
124	51	,	LEX_COMMA	-1	0	-
125	51	i	LEX_ID	9	0	-
126	51	)	LEX_RIGHTTHESIS	-1	0	-
127	51	;	LEX_SEMICOLON	-1	0	-
128	53	t	LEX_SHORT	-1	0	-
129	53	i	LEX_ID	25	0	-
130	53	=	LEX_ASSIGN	-1	0	-
131	53	i	LEX_ID	23	0	-
132	53	(	LEX_LEFTTHESIS	-1	0	-
133	53	i	LEX_ID	7	0	-
134	53	,	LEX_COMMA	-1	0	-
135	53	i	LEX_ID	5	0	-
136	53	)	LEX_RIGHTTHESIS	-1	0	-
137	53	;	LEX_SEMICOLON	-1	0	-
138	57	o	LEX_OUTPUT	-1	0	-

139	57	i	LEX_ID	22	0	-
140	57	;	LEX_SEMICOLON	-1	0	-
141	59	o	LEX_OUTPUT	-1	0	-
142	59	i	LEX_ID	24	0	-
143	59	;	LEX_SEMICOLON	-1	0	-
144	61	o	LEX_OUTPUT	-1	0	-
145	61	i	LEX_ID	25	0	-
146	61	;	LEX_SEMICOLON	-1	0	-
147	65	t	LEX_SHORT	-1	0	-
148	65	i	LEX_ID	26	0	-
149	65	=	LEX_ASSIGN	-1	0	-
150	65	i	LEX_ID	27	0	-
151	65	(	LEX_LEFTTHESIS	-1	0	-
152	65	i	LEX_ID	5	0	-
153	65	)	LEX_RIGHTTHESIS	-1	0	-
154	65	;	LEX_SEMICOLON	-1	0	-
155	67	t	LEX_SHORT	-1	0	-
156	67	i	LEX_ID	28	0	-
157	67	=	LEX_ASSIGN	-1	0	-
158	67	i	LEX_ID	27	0	-
159	67	(	LEX_LEFTTHESIS	-1	0	-
160	67	i	LEX_ID	7	0	-
161	67	)	LEX_RIGHTTHESIS	-1	0	-
162	67	;	LEX_SEMICOLON	-1	0	-
163	69	t	LEX_SHORT	-1	0	-
164	69	i	LEX_ID	29	0	-
165	69	=	LEX_ASSIGN	-1	0	-
166	69	i	LEX_ID	27	0	-
167	69	(	LEX_LEFTTHESIS	-1	0	-
168	69	i	LEX_ID	9	0	-
169	69	)	LEX_RIGHTTHESIS	-1	0	-
170	69	;	LEX_SEMICOLON	-1	0	-
171	73	o	LEX_OUTPUT	-1	0	-
172	73	i	LEX_ID	26	0	-
173	73	;	LEX_SEMICOLON	-1	0	-
174	75	o	LEX_OUTPUT	-1	0	-
175	75	i	LEX_ID	28	0	-
176	75	;	LEX_SEMICOLON	-1	0	-
177	77	o	LEX_OUTPUT	-1	0	-
178	77	i	LEX_ID	29	0	-
179	77	;	LEX_SEMICOLON	-1	0	-
180	81	t	LEX_SHORT	-1	0	-
181	81	i	LEX_ID	30	0	-
182	81	=	LEX_ASSIGN	-1	0	-
183	81	l	LEX_LITERAL	31	0	-
184	81	;	LEX_SEMICOLON	-1	0	-
185	85	w	LEX_SWITCH	-1	0	-

186	85	(	LEX_LEFTTHESIS	-1	0	-
187	85	i	LEX_ID	30	0	-
188	85	)	LEX_RIGHTTHESIS	-1	0	-
189	87	{	LEX_LEFTBRACE	-1	0	-
190	89	c	LEX_CASE	-1	0	-
191	89	l	LEX_LITERAL	32	0	-
192	89	:	LEX_COLON	-1	0	-
193	91	o	LEX_OUTPUT	-1	0	-
194	91	l	LEX_LITERAL	33	0	-
195	91	;	LEX_SEMICOLON	-1	0	-
196	93	k	LEX_BREAK	-1	0	-
197	93	;	LEX_SEMICOLON	-1	0	-
198	97	c	LEX_CASE	-1	0	-
199	97	l	LEX_LITERAL	34	0	-
200	97	:	LEX_COLON	-1	0	-
201	99	o	LEX_OUTPUT	-1	0	-
202	99	l	LEX_LITERAL	35	0	-
203	99	;	LEX_SEMICOLON	-1	0	-
204	101	k	LEX_BREAK	-1	0	-
205	101	;	LEX_SEMICOLON	-1	0	-
206	105	c	LEX_CASE	-1	0	-
207	105	l	LEX_LITERAL	31	0	-
208	105	:	LEX_COLON	-1	0	-
209	107	o	LEX_OUTPUT	-1	0	-
210	107	l	LEX_LITERAL	36	0	-
211	107	;	LEX_SEMICOLON	-1	0	-
212	109	k	LEX_BREAK	-1	0	-
213	109	;	LEX_SEMICOLON	-1	0	-
214	113	c	LEX_CASE	-1	0	-
215	113	l	LEX_LITERAL	37	0	-
216	113	:	LEX_COLON	-1	0	-
217	115	o	LEX_OUTPUT	-1	0	-
218	115	l	LEX_LITERAL	38	0	-
219	115	;	LEX_SEMICOLON	-1	0	-
220	117	k	LEX_BREAK	-1	0	-
221	117	;	LEX_SEMICOLON	-1	0	-
222	121	u	LEX_DEFAULT	-1	0	-
223	121	:	LEX_COLON	-1	0	-
224	123	o	LEX_OUTPUT	-1	0	-
225	123	l	LEX_LITERAL	2	0	-
226	123	;	LEX_SEMICOLON	-1	0	-
227	125	k	LEX_BREAK	-1	0	-
228	125	;	LEX_SEMICOLON	-1	0	-
229	127	}	LEX_RIGHTBRACE	-1	0	-
230	131	r	LEX_RET	-1	0	-
231	131	l	LEX_LITERAL	32	0	-
232	131	;	LEX_SEMICOLON	-1	0	-

```
233   133   }     LEX_RIGHTBRACE  -1      0     -
234   133   $     LEX_UNKNOWN    -1      0     -
```

Листинг В.1 –Таблица лексем

```
===== ID TABLE =====
#     id              data      kind    idxFirst  value
0     main            short     func    1         -
1     a               short     var     6         0
2     L10             short     lit     8         10
3     b               short     var     11        0
4     c               short     var     16        0
5     s1              string    var     21        len=0 ""
6     L1              string    lit     23        len=5
"apple"
7     s2              string    var     26        len=0 ""
8     L2              string    lit     28        len=6
"banana"
9     s3              string    var     31        len=0 ""
10    eq_ab           short     var     36        0
11    ==              -         op      40        -
12    ne_bc           short     var     45        0
13    !=              -         op      49        -
14    lt_ab           short     var     54        0
15    <               -         op      58        -
16    gt_bc           short     var     63        0
17    >               -         op      67        -
18    le_ac           short     var     72        0
19    <=              -         op      76        -
20    ge_cb           short     var     81        0
21    >=              -         op      85        -
22    cmp12           short     var     108       0
23    strcmp          short     func    110       -
24    cmp13           short     var     118       0
25    cmp21           short     var     128       0
26    len1            short     var     147       0
27    strlen          short     func    149       -
28    len2            short     var     155       0
29    len3            short     var     163       0
30    code            short     var     180       0
31    L2_             short     lit     182       2
32    L3              short     lit     190       0
33    L4              string    lit     193       len=6
"case 0"
34    L5              short     lit     198       1
35    L6              string    lit     201       len=6
"case 1"
36    L7              string    lit     209       len=6
```

```
"case 2"
37    L8              short     lit     214       3
38    L9              string    lit     217       len=6
"case 3"
Count: 39
```

Листинг В.2 –Таблица идентификаторов

Приложение Г

```
#include "stdafx.h"
#include "GRB.h"
#include <utility>

namespace GRB
{
    Greibach greibach(
        NS(STARTING_SYMBOL),
        TS('$'),
        {
            Rule(NS(STARTING_SYMBOL), GRB_ERROR_SERIES
+ 0, {
                Rule::Chain({ TS(LEX_SHORT),
TS(LEX_MAIN), TS(LEX_LEFTTHESIS), TS(LEX_RIGHTTHESIS),
NS(BODY) })
            }),

            Rule(NS(BODY), GRB_ERROR_SERIES + 1, {
                Rule::Chain({
                    TS(LEX_LEFTBRACE),
                    NS(STMT_LIST),
                    TS(LEX_RET), NS(ID_OR_LITERAL),
TS(LEX_SEMICOLON),
                    TS(LEX_RIGHTBRACE)
                })
            }),

            Rule(NS(ID_OR_LITERAL), GRB_ERROR_SERIES +
2, {
                Rule::Chain({ TS(LEX_ID) }),
                Rule::Chain({ TS(LEX_LITERAL) })
            }),

            Rule(NS(END_STMT), GRB_ERROR_SERIES + 3, {
                Rule::Chain({ TS(LEX_SEMICOLON) })
            }),

                // -------------------- STATEMENTS ---
-----------------
                Rule(NS(STMT_LIST), GRB_ERROR_SERIES +
10, {
                    Rule::Chain({ TS(LEX_SHORT),
TS(LEX_ID), NS(DECL_TAIL), NS(STMT_LIST) }),
                    Rule::Chain({ TS(LEX_STRING),
TS(LEX_ID), NS(DECL_TAIL), NS(STMT_LIST) }),
```

```
                    Rule::Chain({ TS(LEX_ID),
NS(ID_START), NS(STMT_LIST) }),

                    Rule::Chain({ TS(LEX_OUTPUT),
NS(EXPR), NS(END_STMT), NS(STMT_LIST) }),
                    Rule::Chain({ TS(LEX_SWITCH),
NS(SWITCH_TAIL_STMT), NS(STMT_LIST) }),
                    Rule::Chain({ TS(LEX_BREAK),
NS(END_STMT), NS(STMT_LIST) }),

                    // stop markers
                    Rule::Chain({ TS(LEX_RET),
TS(EMPTY_CHAIN) }),
                    Rule::Chain({ TS(LEX_RIGHTBRACE),
TS(EMPTY_CHAIN) }),

                    // IMPORTANT for switch-case: stop
stmt list before next case/default
                    Rule::Chain({ TS(LEX_CASE),
TS(EMPTY_CHAIN) }),
                    Rule::Chain({ TS(LEX_DEFAULT),
TS(EMPTY_CHAIN) }),

                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                Rule(NS(ID_START), GRB_ERROR_SERIES +
11, {
                    Rule::Chain({ TS(LEX_ASSIGN),
NS(EXPR), NS(END_STMT) }),
                    Rule::Chain({ TS(LEX_LEFTTHESIS),
NS(CALL_PARAMS_OPT), TS(LEX_RIGHTTHESIS), NS(END_STMT)
})
                }),

                Rule(NS(DECL_TAIL), GRB_ERROR_SERIES +
21, {
                    Rule::Chain({ TS(LEX_SEMICOLON)
}),
                    Rule::Chain({ TS(LEX_ASSIGN),
NS(INIT_TAIL) })
                }),

                Rule(NS(INIT_TAIL), GRB_ERROR_SERIES +
23, {
                    Rule::Chain({ TS(LEX_ID),
```

```
NS(ID_TAIL), NS(LOGICAL_EXPR), NS(END_STMT) }),
                    Rule::Chain({ TS(LEX_LITERAL),
NS(LOGICAL_EXPR), NS(END_STMT) }),
                    Rule::Chain({ TS(LEX_LEFTTHESIS),
NS(EXPR), TS(LEX_RIGHTTHESIS), NS(LOGICAL_EXPR),
NS(END_STMT) })
                }),

                // -------------------- EXPRESSIONS --
------------------
                Rule(NS(EXPR), GRB_ERROR_SERIES + 80,
{
                    Rule::Chain({ TS(LEX_ID),
NS(ID_TAIL), NS(LOGICAL_EXPR) }),
                    Rule::Chain({ TS(LEX_LITERAL),
NS(LOGICAL_EXPR) }),
                    Rule::Chain({ TS(LEX_LEFTTHESIS),
NS(EXPR), TS(LEX_RIGHTTHESIS), NS(LOGICAL_EXPR) })
                }),

                Rule(NS(TERM), GRB_ERROR_SERIES + 81,
{
                    Rule::Chain({ TS(LEX_ID),
NS(ID_TAIL) }),
                    Rule::Chain({ TS(LEX_LITERAL) }),
                    Rule::Chain({ TS(LEX_LEFTTHESIS),
NS(EXPR), TS(LEX_RIGHTTHESIS) })
                }),

                Rule(NS(LOGICAL_EXPR),
GRB_ERROR_SERIES + 82, {
                    Rule::Chain({ TS(LEX_OPERATOR),
NS(TERM), NS(LOGICAL_EXPR) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                Rule(NS(ID_TAIL), GRB_ERROR_SERIES +
84, {
                    Rule::Chain({ TS(LEX_LEFTTHESIS),
NS(CALL_PARAMS_OPT), TS(LEX_RIGHTTHESIS) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                // Z: id M | literal M | e
(deterministic by first terminal)
                Rule(NS(CALL_PARAMS_OPT),
GRB_ERROR_SERIES + 91, {
```

```
                    Rule::Chain({ TS(LEX_ID),
NS(PARAM_TAIL) }),
                    Rule::Chain({ TS(LEX_LITERAL),
NS(PARAM_TAIL) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                Rule(NS(PARAM_TAIL), GRB_ERROR_SERIES
+ 92, {
                    Rule::Chain({ TS(LEX_COMMA),
TS(LEX_ID),      NS(PARAM_TAIL) }),
                    Rule::Chain({ TS(LEX_COMMA),
TS(LEX_LITERAL), NS(PARAM_TAIL) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                // -------------------- SWITCH -------
-------------
                Rule(NS(SWITCH_TAIL_STMT),
GRB_ERROR_SERIES + 100, {
                    Rule::Chain({
                        TS(LEX_LEFTTHESIS), NS(EXPR),
TS(LEX_RIGHTTHESIS),
                        TS(LEX_LEFTBRACE),
NS(CASE_LIST), NS(DEFAULT_CASE),
                        TS(LEX_RIGHTBRACE)
                    })
                }),

                Rule(NS(CASE_LIST), GRB_ERROR_SERIES +
110, {
                    Rule::Chain({ TS(LEX_CASE),
TS(LEX_LITERAL), TS(LEX_COLON), NS(CASE_BLOCK),
NS(CASE_LIST) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),

                Rule(NS(CASE_BLOCK), GRB_ERROR_SERIES
+ 112, {
                    Rule::Chain({ NS(STMT_LIST),
TS(LEX_BREAK), NS(END_STMT) }),
                    Rule::Chain({ TS(LEX_BREAK),
NS(END_STMT) }),
                    Rule::Chain({ NS(STMT_LIST) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),
```

```
                Rule(NS(DEFAULT_CASE),
GRB_ERROR_SERIES + 120, {
                    Rule::Chain({ TS(LEX_DEFAULT),
TS(LEX_COLON), NS(CASE_BLOCK) }),
                    Rule::Chain({ TS(EMPTY_CHAIN) })
                }),
        }
    );

    // ================== Chain ==================

Rule::Chain::Chain(std::initializer_list<GRBALPHABET>
init)
    {
        size = (short)init.size();
        nt = (size > 0) ? new GRBALPHABET[size] :
nullptr;
        int i = 0;
        for (auto v : init) nt[i++] = v;
    }

    Rule::Chain::~Chain()
    {
        delete[] nt;
        nt = nullptr;
        size = 0;
    }

    Rule::Chain::Chain(const Chain& other)
    {
        size = other.size;
        nt = (size > 0) ? new GRBALPHABET[size] :
nullptr;
        for (int i = 0; i < size; ++i) nt[i] =
other.nt[i];
    }

    Rule::Chain& Rule::Chain::operator=(const Chain&
other)
    {
        if (this == &other) return *this;
        delete[] nt;
        size = other.size;
        nt = (size > 0) ? new GRBALPHABET[size] :
nullptr;
        for (int i = 0; i < size; ++i) nt[i] =
other.nt[i];
```

```
        return *this;
    }

    Rule::Chain::Chain(Chain&& other) noexcept
        : size(other.size), nt(other.nt)
    {
        other.size = 0;
        other.nt = nullptr;
    }

    Rule::Chain& Rule::Chain::operator=(Chain&& other)
noexcept
    {
        if (this == &other) return *this;
        delete[] nt;
        size = other.size;
        nt = other.nt;
        other.size = 0;
        other.nt = nullptr;
        return *this;
    }

    char* Rule::Chain::getCChain(char* b)
    {
        for (int i = 0; i < size; ++i)
            b[i] = (nt[i] == EMPTY_CHAIN) ? 'e' :
Chain::alphabet_to_char(nt[i]);
        b[size] = 0x00;
        return b;
    }
    Rule::Rule(GRBALPHABET pnn, int piderror,
std::initializer_list<Chain> chainsInit)
        : nn(pnn), iderror(piderror), size(0),
chains(nullptr)
    {
        size = (short)chainsInit.size();
        chains = (size > 0) ? new Chain[size] :
nullptr;

        int i = 0;
        for (auto ch : chainsInit) chains[i++] = ch;
    }

    Rule::~Rule()
    {
        delete[] chains;
        chains = nullptr;
```

```
        size = 0;
    }

    Rule::Rule(const Rule& other)
        : nn(other.nn), iderror(other.iderror),
size(other.size), chains(nullptr)
    {
        chains = (size > 0) ? new Chain[size] :
nullptr;
        for (int i = 0; i < size; ++i) chains[i] =
other.chains[i];
    }

    Rule& Rule::operator=(const Rule& other)
    {
        if (this == &other) return *this;
        delete[] chains;

        nn = other.nn;
        iderror = other.iderror;
        size = other.size;

        chains = (size > 0) ? new Chain[size] :
nullptr;
        for (int i = 0; i < size; ++i) chains[i] =
other.chains[i];
        return *this;
    }

    Rule::Rule(Rule&& other) noexcept
        : nn(other.nn), iderror(other.iderror),
size(other.size), chains(other.chains)
    {
        other.size = 0;
        other.chains = nullptr;
        other.nn = 0;
        other.iderror = 0;
    }

    Rule& Rule::operator=(Rule&& other) noexcept
    {
        if (this == &other) return *this;
        delete[] chains;

        nn = other.nn;
        iderror = other.iderror;
        size = other.size;
```

```
        chains = other.chains;

        other.size = 0;
        other.chains = nullptr;
        other.nn = 0;
        other.iderror = 0;
        return *this;
    }

    char* Rule::getCRule(char* b, short nchain)
    {
        char bchain[200];
        b[0] = Chain::alphabet_to_char(nn);
        b[1] = '-';
        b[2] = '>';
        b[3] = 0x00;
        chains[nchain].getCChain(bchain);
        strcat_s(b, 200, bchain);
        return b;
    }

    short Rule::getNextChain(GRBALPHABET t,
Rule::Chain& pchain, short j)
    {
        auto firstTerminal = [](const Rule::Chain&
ch)->GRBALPHABET {
            for (int i = 0; i < ch.size; ++i)
                if (Rule::Chain::isT(ch.nt[i])) return
ch.nt[i];
            return TS(EMPTY_CHAIN);
            };

        short k = j;
        while (k < size && firstTerminal(chains[k]) !=
t) ++k;
        if (k < size) { pchain = chains[k]; return k;
}

        k = j;
        while (k < size && !(chains[k].size == 1 &&
chains[k].nt[0] == TS(EMPTY_CHAIN))) ++k;
        if (k < size) { pchain = chains[k]; return k;
}

        return -1;
    }
```

```
    Greibach::Greibach(GRBALPHABET pstartN,
GRBALPHABET pstbottom, std::initializer_list<Rule>
rulesInit)
        : size(0), startN(pstartN),
stbottomT(pstbottom), rules(nullptr)
    {
        size = (short)rulesInit.size();
        rules = (size > 0) ? new Rule[size] : nullptr;

        int i = 0;
        for (auto r : rulesInit) rules[i++] = r;
    }
    Greibach::~Greibach()
    {
        delete[] rules;
        rules = nullptr;
        size = 0;
    }
    Greibach::Greibach(const Greibach& other)
        : size(other.size), startN(other.startN),
stbottomT(other.stbottomT), rules(nullptr)
    {
        rules = (size > 0) ? new Rule[size] : nullptr;
        for (int i = 0; i < size; ++i) rules[i] =
other.rules[i];
    }

    Greibach& Greibach::operator=(const Greibach&
other)
    {
        if (this == &other) return *this;
        delete[] rules;

        size = other.size;
        startN = other.startN;
        stbottomT = other.stbottomT;

        rules = (size > 0) ? new Rule[size] : nullptr;
        for (int i = 0; i < size; ++i) rules[i] =
other.rules[i];
        return *this;
    }

    Greibach::Greibach(Greibach&& other) noexcept
        : size(other.size), startN(other.startN),
stbottomT(other.stbottomT), rules(other.rules)
    {
```

```
        other.size = 0;
        other.startN = 0;
        other.stbottomT = 0;
        other.rules = nullptr;
    }

    Greibach& Greibach::operator=(Greibach&& other)
noexcept
    {
        if (this == &other) return *this;
        delete[] rules;

        size = other.size;
        startN = other.startN;
        stbottomT = other.stbottomT;
        rules = other.rules;

        other.size = 0;
        other.startN = 0;
        other.stbottomT = 0;
        other.rules = nullptr;
        return *this;
    }

    short Greibach::getRule(GRBALPHABET pnn, Rule&
prule)
    {
        for (short k = 0; k < size; ++k)
            if (rules[k].nn == pnn) { prule =
rules[k]; return k; }
        return -1;
    }

    Rule Greibach::getRule(short n)
    {
        Rule rc;
        if (n < size) rc = rules[n];
        return rc;
    }

    Greibach getGreibach()
    {
        return greibach;
    }
}
```

Листинг Г.1 – Реализация GRB.cpp

Приложение Д

```
0     : S->tm()B
1     : B->{QrN;}
2     : Q->tiJQ
3     : J->=I
4     : I->lLX
5     : L->e
6     : X->;
7     : Q->giJQ
8     : I->(E)LX
9     : E->iyL
10    : y->e
11    : L->pTL
12    : T->iy
13    : Q->oEXQ
14    : I->iyLX
15    : y->(Z)
16    : Z->iM
17    : M->,iM
18    : M->e
19    : Q->wHQ
20    : H->(E){CD}
```

Листинг Д.1 – Дерево разбора

Приложение Е

```
#include "PolishNotation.h"
#include <stack>
#include <queue>
#include <vector>
#include <cstring>

#include "../StandartLib/StandartLib.h"

namespace Polish {

    // создаёт/находит literal Pn (число параметров) в
idtable
    static int ensureParamCountLiteral(int value,
IT::IdTable& idtable)
    {
        for (int k = 0; k < idtable.size; ++k) {
            if (idtable.table[k].idtype == IT::L &&
                idtable.table[k].idDataType == IT::SHORT
&&
                idtable.table[k].value.vint == value)
                return k;
        }

        IT::Entry e{};
        e.idtype = IT::L;
        e.idDataType = IT::SHORT;
        e.value.vint = value;
        e.idxFirstLE = TI_NULLIDX;

        char name[ID_MAXSIZE]{};
        strcpy_s(name, "P");
        char num[16]{};
        _itoa_s(value, num, 10);
        strcat_s(name, num);
        strcpy_s(e.id, name);

        // на всякий случай делаем имя уникальным
        if (IT::IsId(idtable, e.id) != TI_NULLIDX) {
            strcat_s(e.id, "_");
        }

        IT::Add(idtable, e);
        return idtable.size - 1;
    }
```

```
    // найти конец выражения (точку с запятой)
    static int findExprEnd(int from, Lex::LEX& lex)
    {
        for (int j = from; j < lex.lextable.size; ++j) {
            if (lex.lextable.table[j].lexema ==
LEX_SEMICOLON) return j;
        }
        return -1;
    }

    static void writeQueueToRange(int from, int
toExclusive, Lex::LEX& lex, std::queue<LT::Entry>& q)
    {
        int pos = from;

        // 1) пишем ОПЗ
        while (pos < toExclusive && !q.empty()) {
            lex.lextable.table[pos++] = q.front();
            q.pop();
        }

        // 2) остаток диапазона добиваем '#', чтобы ни
семантика, ни генератор не видели "хвост инфикса"
        while (pos < toExclusive) {
            LT::Entry pad{};
            pad.lexema = '#';                  // важное:
именно '#'
            pad.idxTI = LT_TI_NULLIDX;
            pad.line = lex.lextable.table[from].line;
            pad.op = LT::ONOT;
            pad.priority = 0;
            lex.lextable.table[pos++] = pad;
        }
    }



    bool polishNotation(int from, Lex::LEX& lex)
    {
        // границы выражения
        const int semi = findExprEnd(from, lex);
        if (semi < 0) return false;

        std::stack<LT::Entry> opStack;
        std::queue<LT::Entry> outQ;

        // режим обработки std-функции
```

```
        bool isStdFuncCall = false;
        int  idxFunc = LT_TI_NULLIDX;
        int  commaCount = 0;
        bool hadAnyArg = false;

        // шунтирующая станция по диапазону [from..semi-
1]
        for (int i = from; i < semi; ++i)
        {
            LT::Entry cur = lex.lextable.table[i]; //
копия, чтобы не портить исходник по месту

            switch (cur.lexema)
            {
            case LEX_ID:
            {
                // если это std-функция — включаем режим
вызова и НЕ кладём имя как операнд
                if (cur.idxTI != LT_TI_NULLIDX) {
                    const char* nm =
lex.idtable.table[cur.idxTI].id;
                    if (StandartLib::IsStdFunction(nm)) {
                        isStdFuncCall = true;
                        idxFunc = cur.idxTI;
                        commaCount = 0;
                        hadAnyArg = false;
                        break; // continue цикла
                    }
                }

                // обычный идентификатор = операнд
                outQ.push(cur);
                if (isStdFuncCall) hadAnyArg = true;
                break;
            }

            case LEX_LITERAL:
                outQ.push(cur);
                if (isStdFuncCall) hadAnyArg = true;
                break;

            case LEX_COMMA:
                // разделитель аргументов функции:
                // выгружаем операторы до ближайшей '('
                if (isStdFuncCall) {
                    ++commaCount;
                    while (!opStack.empty() &&
```

```
opStack.top().lexema != LEX_LEFTTHESIS) {
                        outQ.push(opStack.top());
                        opStack.pop();
                    }
                }
                break;

            case LEX_LEFTTHESIS:
                opStack.push(cur);
                break;

            case LEX_RIGHTTHESIS:
                // выгружаем всё до '('
                while (!opStack.empty() &&
opStack.top().lexema != LEX_LEFTTHESIS) {
                    outQ.push(opStack.top());
                    opStack.pop();
                }
                if (opStack.empty()) return false; // нет
'('
                opStack.pop(); // снимаем '('

                // если это было закрытие вызова std-
функции — добавляем маркер вызова + Pn
                if (isStdFuncCall) {
                    LT::Entry call{};
                    call.lexema = LEX_CALL;
                    call.idxTI = idxFunc;
                    call.line = cur.line;
                    call.op = LT::ONOT;
                    call.priority = 0;
                    outQ.push(call);

                    const int realCount = hadAnyArg ?
(commaCount + 1) : 0;
                    const int litIdx =
ensureParamCountLiteral(realCount, lex.idtable);

                    LT::Entry cnt{};
                    cnt.lexema = LEX_LITERAL;
                    cnt.idxTI = litIdx;
                    cnt.line = cur.line;
                    cnt.op = LT::ONOT;
                    cnt.priority = 0;
                    outQ.push(cnt);

                    // сброс режима
```

```
                    isStdFuncCall = false;
                    idxFunc = LT_TI_NULLIDX;
                    commaCount = 0;
                    hadAnyArg = false;
                }
                break;

            case LEX_OPERATOR:
                // классический shunting-yard: пока на
стеке оператор с >= приоритетом — выгружаем
                while (!opStack.empty()
                    && opStack.top().lexema ==
LEX_OPERATOR
                    && cur.priority <=
opStack.top().priority)
                {
                    outQ.push(opStack.top());
                    opStack.pop();
                }
                opStack.push(cur);
                break;

            default:
                // всё прочее внутри выражения игнорируем
                break;
            }
        }

        // выгружаем остаток стека
        while (!opStack.empty()) {
            if (opStack.top().lexema == LEX_LEFTTHESIS ||
                opStack.top().lexema == LEX_RIGHTTHESIS)
                return false;
            outQ.push(opStack.top());
            opStack.pop();
        }

        // перезаписываем выражение в LT в том же
диапазоне [from..semi-1]
        writeQueueToRange(from, semi, lex, outQ);

        return true;
    }

    void startPolish(Lex::LEX& lex)
    {
        for (int i = 0; i < lex.lextable.size; ++i) {
```

```
            if (lex.lextable.table[i].lexema ==
LEX_ASSIGN) {
                // выражение после '='
                polishNotation(i + 1, lex);
            }
            else if (lex.lextable.table[i].lexema ==
LEX_OUTPUT) {
                // выражение внутри output(...)
                polishNotation(i + 1, lex);
            }
            else if (lex.lextable.table[i].lexema ==
LEX_ID &&
                lex.lextable.table[i].idxTI !=
LT_TI_NULLIDX &&

lex.idtable.table[lex.lextable.table[i].idxTI].idDataType
== IT::PROC)
            {
                // вызов процедуры (если ты так
кодируешь)
                polishNotation(i, lex);
            }
        }

        // синхронизацию — один раз после всех
перестановок
        Lex::Synchronization(lex);
    }

}
```

Листинг Е.1 – Реализация преобразования PolishNotation.cpp

Приложение Ж

```
function output(x) { console.log(x); }

const L10 = 10;
const L1 = "apple";
const L2 = "banana";
const L2_ = 2;
const L3 = 0;
const L4 = "case 0";
const L5 = 1;
const L6 = "case 1";
const L7 = "case 2";
const L8 = 3;
const L9 = "case 3";
let a = 0;
let b = 0;
let c = 0;
let s1 = "";
let s2 = "";
let s3 = "";
let eq_ab = 0;
let ne_bc = 0;
let lt_ab = 0;
let gt_bc = 0;
let le_ac = 0;
let ge_cb = 0;
let cmp12 = 0;
let cmp13 = 0;
let cmp21 = 0;
let len1 = 0;
let len2 = 0;
let len3 = 0;
let code = 0;

function main() {
a = L10;
b = L10;
c = L10;
s1 = L1;
s2 = L2;
s3 = L1;
eq_ab = (a == b);
ne_bc = (b != c);
lt_ab = (a < b);
gt_bc = (b > c);
le_ac = (a <= c);
```

```
ge_cb = (c >= b);
output(eq_ab);
output(ne_bc);
output(lt_ab);
output(gt_bc);
output(le_ac);
output(ge_cb);
cmp12 = (s1).localeCompare(s2);
cmp13 = (s1).localeCompare(s3);
cmp21 = (s2).localeCompare(s1);
output(cmp12);
output(cmp13);
output(cmp21);
len1 = (s1).length;
len2 = (s2).length;
len3 = (s3).length;
output(len1);
output(len2);
output(len3);
code = L2_;
switch (code) {
  case 0:
output(L4);
    break;
  case 1:
output(L6);
    break;
  case 2:
output(L7);
    break;
  case 3:
output(L9);
    break;
  default:
output(L10);
    break;
}
}

main();
```

Листинг Е.1 – Результат генерации исходного кода в JavaScript